

Método de la Ingeniería

Proyecto: Sokoban — Tarea Integradora

Curso: Computación y Estructuras Discretas I

Equipo: Herrera · Fernández · Quintero

Fecha: 25 de abril de 2026

1. Planteamiento del problema

El juego Sokoban necesita gestionar cuatro situaciones que surgen naturalmente durante una partida y que ninguna variable simple puede resolver:

- El jugador puede equivocarse y necesita regresar al estado anterior movimiento a movimiento.
- Si el usuario presiona varias teclas muy seguidas, el juego no puede ignorar ninguna — deben procesarse en el orden exacto en que se ingresaron.
- Al terminar cada nivel se guarda el resultado de la partida, y el sistema debe tener siempre disponibles los mejores resultados sin reordenar todo cada vez.
- Durante la búsqueda automática de solución (nivel experto), el algoritmo puede llegar al mismo tablero por distintos caminos. Sin detección de estados repetidos, entraría en ciclos infinitos.

Cada una de estas situaciones tiene un patrón de acceso a datos diferente. El objetivo es identificar cuál es ese patrón en cada caso y seleccionar la estructura que lo satisface de forma óptima.

Restricciones relevantes del sistema:

- No se pueden usar estructuras de `java.util` — todo debe ser implementación propia con listas enlazadas.
- Las operaciones críticas del motor de juego (registrar movimiento, deshacer, procesar tecla) deben ocurrir en tiempo constante para no afectar la fluidez.

- El estado del tablero en cualquier momento queda determinado únicamente por: posición del jugador + posición de cada caja. Los muros son fijos.
-

2. Análisis

El primer paso es descomponer cada situación en su operación esencial y preguntarse qué elemento de la estructura necesita ser accesible y cuándo.

Deshacer movimientos

Cada vez que el jugador se mueve, el sistema guarda una fotografía del estado actual. Cuando el jugador presiona "deshacer", necesita recuperar la fotografía más reciente. Si vuelve a presionar "deshacer", necesita la anterior a esa. El patrón es claro: siempre se accede al elemento que entró de último. Nunca se necesita el primero, ni el del medio, ni ninguno por índice.

Buffer de entrada

Los comandos del teclado deben ejecutarse en el mismo orden en que se presionaron. Si el jugador presionó arriba, derecha, abajo en secuencia rápida, el juego debe ejecutarlos en ese orden. El primero que entró debe ser el primero en procesarse. Es el patrón opuesto al anterior.

Leaderboard

Al terminar un nivel se inserta un resultado nuevo. Esta operación ocurre pocas veces (una por partida jugada). En cambio, mostrar el mejor resultado es una operación frecuente. Conviene pagar el costo en la inserción — que ocurre poco — para que la consulta del mejor elemento sea inmediata.

Detección de estados visitados

El algoritmo de búsqueda necesita saber, dado un estado del tablero, si ya lo exploró antes. La pregunta es siempre la misma: "¿existe este estado en el conjunto de visitados?". No importa el orden, no importa el índice, no importa el mínimo. Solo importa si existe o no, y la respuesta debe ser lo más rápida posible porque se consulta miles de veces durante una búsqueda BFS o DFS.

Resumen de patrones identificados:

Situación	Elemento que siempre se necesita	Patrón
Undo	El más recientemente insertado	LIFO
Input buffer	El que lleva más tiempo esperando	FIFO
Leaderboard	El de mayor prioridad (mejor score)	Prioridad
Estados visitados	Cualquiera, solo verificar existencia	Lookup por clave

3. Modelado

Con los patrones identificados, evaluamos qué estructuras los satisfacen y descartamos las que no.

3.1 Undo → Pila

Operaciones requeridas:

- `guardar(estado)` — al ejecutar cada movimiento
- `recuperar()` — al presionar deshacer, retorna el estado más reciente

¿Por qué una Pila?

La Pila es la única estructura que garantiza acceso al último elemento insertado en $O(1)$ de forma natural, sin búsqueda. En una lista desordenada habría que recorrerla hasta el final. En un array sin puntero al tope habría que rastrear el índice. La Pila mantiene un puntero `top` que siempre apunta al elemento más reciente — exactamente lo que necesitamos.

¿Por qué no una Cola? Porque la Cola entrega el elemento más antiguo, que es lo contrario de lo que queremos.

¿Por qué no un Array? Funcionaría, pero requiere manejar manualmente el índice del tope y redimensionar cuando se llena. La Pila con lista enlazada crece ilimitadamente sin ese costo.

Especificación TAD:

ESTRUCTURA: Pila<GameSnapshot>

push(s: GameSnapshot)

Pre: $s \neq \text{null}$

Post: $\text{top} = s$

$\text{size} = \text{size_anterior} + 1$

pop() → GameSnapshot

Pre: $\text{isEmpty}() = \text{false}$

Post: retorna el GameSnapshot insertado más recientemente

$\text{size} = \text{size_anterior} - 1$

peek() → GameSnapshot

Pre: $\text{isEmpty}() = \text{false}$

Post: retorna top sin modificar la estructura

Invariante: en todo momento, pop() retorna los elementos

en orden inverso al que fueron insertados con push()

Uso en el juego: antes de ejecutar cada movimiento, `Game.move()` hace `undoStack.push(new GameSnapshot(...))`. Si el movimiento resulta inválido, el snapshot se descarta con `pop()`. Si el jugador presiona deshacer, `undo()` llama `pop()` y restaura el estado.

3.2 Input Buffer → Cola

Operaciones requeridas:

- `agregar(comando)` — cada vez que el usuario presiona una tecla

- `procesar()` — extrae el comando más antiguo para ejecutarlo

¿Por qué una Cola?

La Cola garantiza que el primer elemento en entrar es el primero en salir (FIFO). Con punteros `front` y `rear`, tanto la inserción al final como la extracción del frente son $O(1)$. Ninguna otra estructura logra esto sin costo adicional.

¿Por qué no una Pila? Invertiría el orden de los comandos — el último en presionar sería el primero en ejecutarse, que es exactamente lo que queremos evitar.

¿Por qué no una Lista? Una lista sin puntero `front` dedicado requiere recorrer hasta el primer elemento para extraerlo, costando $O(n)$. Con la Cola se tiene acceso directo en $O(1)$.

Especificación TAD:

ESTRUCTURA: Cola<Direction>

enqueue(d: Direction)

Pre: $d \in \{UP, DOWN, LEFT, RIGHT\}$

Post: d queda al final de la cola

$size = size_anterior + 1$

dequeue() $\rightarrow$ Direction

Pre: `isEmpty()` = false

Post: retorna el Direction que lleva más tiempo en la cola

$size = size_anterior - 1$

Invariante: el orden de extracción es idéntico al orden de inserción.

Si se insertaron d_1, d_2, d_3 en ese orden,

`dequeue()` retorna d_1 , luego d_2 , luego d_3 .

Uso en el juego: `Game.queueCommand(direction)` encola el comando.

`Game.processInputBuffer()` llama `dequeue()` en ciclo hasta vaciar la cola, ejecutando cada movimiento en orden.

3.3 Leaderboard → Cola de Prioridad

Operaciones requeridas:

- `insertar(resultado)` — al terminar cada nivel (poco frecuente)
- `consultarMejor()` — para mostrar el ranking (frecuente)

¿Por qué una Cola de Prioridad?

El requisito central es que el mejor resultado siempre esté disponible de inmediato. Una Cola de Prioridad implementada como lista enlazada ordenada coloca el elemento de mayor prioridad en `head`, logrando consulta en $O(1)$. El costo se paga en la inserción ($O(n)$), que ocurre pocas veces.

Comparación de opciones:

Lista desordenada: insertar $O(1)$, pero consultar el mejor requiere recorrer toda la lista $O(n)$. Malo para consultas frecuentes.

Lista ordenada manualmente: cada inserción requiere $O(n)$ para encontrar el lugar correcto y $O(n)$ para mover elementos si es array. La Cola de Prioridad hace exactamente esto pero de forma estructurada y con nombre semántico claro.

Árbol BST: también daría $O(\log n)$ en inserción y consulta, pero es más complejo de implementar y para el volumen de datos del leaderboard (pocos registros por nivel) el beneficio no justifica la complejidad.

Función de prioridad:

El score de cada partida se calcula como:

$$\text{score}(s) = (1000 / \text{movimientos}) + (500 / (\text{empujes} + 1)) + (2000 / \text{tiempo})$$

Un score más alto indica mejor partida: menos movimientos, menos empujes, menos tiempo. El elemento con score más alto queda en `head` y es el primero en extraerse.

Especificación TAD:

ESTRUCTURA: ColaPrioridad<Stats>

insert(s: Stats)

Pre: $s \neq \text{null} \wedge s.\text{isCompleted}() = \text{true}$

Post: s queda posicionado de manera que

$\text{head.score} \geq s.\text{score}$ para todo elemento antes de s

extractMin() $\rightarrow$ Stats

Pre: isEmpty() = false

Post: retorna el Stats con el mayor score (mejor partida)

size = size_anterior - 1

Invariante: $\forall$ elemento e en la estructura,

$\text{head.score} \geq e.\text{score}$

3.4 Estados Visitados $\rightarrow$ Tabla Hash

Operaciones requeridas:

- `marcar(estado)` — al explorar un nuevo estado durante la búsqueda
- `yaVisitado?(estado)` — antes de explorar cada estado candidato

¿Por qué una Tabla Hash?

Durante una búsqueda BFS o DFS sobre el espacio de estados de Sokoban, el algoritmo puede generar decenas de miles de estados. La operación `yaVisitado?` se ejecuta en cada paso. Si esta operación costara $O(n)$, con n estados visitados, el costo total sería $O(n^2)$ — inaceptable para el nivel experto.

La Tabla Hash responde `containsKey` en $O(1)$ promedio, independientemente de cuántos estados haya almacenado. Esto mantiene el costo total de la búsqueda en $O(n)$ por las consultas, en lugar de $O(n^2)$.

Representación del estado como clave:

Cada estado del tablero se codifica como un string único:

key = "fila_jugador,col_jugador;fila_caja₁,col_caja₁|fila_caja₂,col_caja₂|..."

Ejemplo con jugador en (3,2) y dos cajas en (3,4) y (5,6):

key = "3,2;3,4|5,6|"

Esta clave es inyectiva: dos estados diferentes del tablero producen claves diferentes, y el mismo estado siempre produce la misma clave. Los muros no se incluyen porque son fijos en todos los estados.

Función hash utilizada:

$$h(\text{key}) = |\sum_i \text{key}[i] \times 31^{(k-i-1)}| \bmod \text{capacity}$$

Base 31 porque es primo, lo que distribuye los caracteres de forma más uniforme entre los buckets y reduce la probabilidad de colisiones. Es la misma base que usa `String.hashCode()` en Java internamente.

Las colisiones se resuelven con **encadenamiento separado**: cada bucket contiene una lista enlazada de entradas que mapearon al mismo índice. Con factor de carga < 0.75 (umbral de redimensionamiento), la longitud esperada de cada cadena es menor que 1, manteniendo el $O(1)$ promedio.

Especificación TAD:

ESTRUCTURA: `TablaHash<String, Boolean>`

`put(key: String, value: Object)`

Pre: `key ≠ null`

Post: `containsKey(key) = true`

`get(key) = value`

si key era nueva: `size = size_anterior + 1`

`containsKey(key: String) → Boolean`

Pre: `key ≠ null`

Post: `retorna true ↔ key fue insertada previamente con put()`

Invariante: $\text{factor_de_carga} = \text{size} / \text{capacity} < 0.75$

Si se supera, `resize()` duplica `capacity` y rehashea todo.

4. Solución

La asignación final de estructuras al sistema:

Problema del juego	Estructura	Clase implementada
Deshacer movimientos	Pila	<code>CustomStack<GameSnapshot></code>
Buffer de entrada del teclado	Cola	<code>CustomQueue<Direction></code>
Leaderboard por nivel	Cola de Prioridad	<code>CustomPriorityQueue<Stats></code>
Detección de estados visitados	Tabla Hash	<code>TranspositionTable</code>

5. Conclusión

El análisis mostró que las cuatro situaciones del juego tienen patrones de acceso fundamentalmente distintos, lo que hace que ninguna estructura sirva para todas:

La Pila resuelve el undo porque su naturaleza LIFO coincide exactamente con "necesito lo más reciente primero". La Cola resuelve el buffer de entrada porque su naturaleza FIFO preserva el orden temporal de los comandos. La Cola de Prioridad resuelve el leaderboard porque concentra el costo en la inserción (infrecuente) y garantiza acceso inmediato al mejor resultado (frecuente). La Tabla Hash resuelve la detección de ciclos porque es la única estructura que responde preguntas de existencia en $O(1)$ independientemente del tamaño del conjunto.

La decisión más crítica es la tabla hash para la transposición. Sin ella, el algoritmo de búsqueda del nivel experto consultaría estados visitados en $O(n)$ en cada paso, elevando el costo total a $O(n^2)$. Con miles de estados posibles en un tablero de Sokoban, esto haría la búsqueda impracticable. La tabla hash no es una optimización opcional sino un requisito para que el nivel experto funcione correctamente.
